

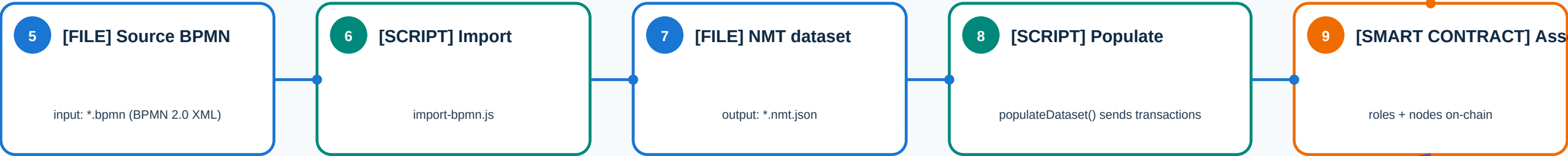
# BPMN -> NMT -> Contract -> BPMN

One complete flow: import a choreography, persist it in NMT, then render it again

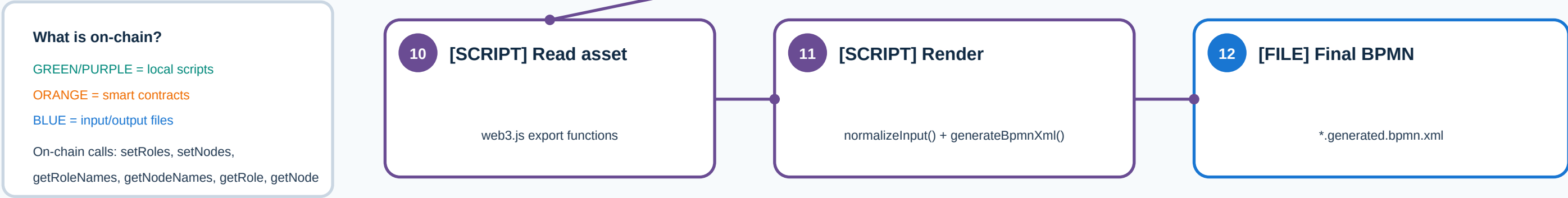
## A. LOCAL SCRIPTS



## B. IMPORT + STORE



## C. EXPORT + RENDER



### What is on-chain?

GREEN/PURPLE = local scripts

ORANGE = smart contracts

BLUE = input/output files

On-chain calls: setRoles, setNodes,  
getRoleNames, getNodeNames, getRole, getNode

Run the whole import-to-contract-to-render sequence with:

```
npm run flow:import-bpmn -- <asset-address> <input.bpmn>
```

# Execution checklist and artifacts

Use a new asset address for each independent BPMN import test

1

## Install dependencies

`npm install`

`cd bpmn-builder-js && npm install && cd ..`

2

## Compile contracts

`npm run compile`

Creates Solidity artifacts used by `deploy-local.js`

3

## Start local blockchain

`npm run node`

Leave this command running in its own terminal

4

## Deploy and mint an asset

`npm run deploy:local`

Copy: `ChoreographyMutableAsset` minted at: `0x...`

5

## Run the complete flow

`npm run flow:import-bpmn -- <asset-address> <input.bpmn>`

Imports, stores, exports, normalizes and renders

### Artifacts created by the final command

- Imported NMT dataset: `bpmn-builder-js/example/input/<name>.nmt.json`
- Raw contract export: `bpmn-builder-js/example/input/<name>-from-contract.raw.generated.json`
- Normalized render input: `bpmn-builder-js/example/input/<name>-from-contract.normalized.generated.json`
- Final rendered diagram: `bpmn-builder-js/example/output/<name>-from-contract.generated.bpmn.xml`